Sentiment Analysis

Analyzing Customer Reviews from Yelp

Group 5: Esbeide Ibarra, Joel Garza, Nhat Nam Khanh Nguyen
Professor: Viswanatha Rao

The Problem

🔴 Challenge: Businesses receive thousands of customer reviews monthly

❓ Question: How can we automatically understand customer sentiment?

⏱️ Issue: Manual review analysis is time-consuming (~1 hour per 100 reviews)

💡 Opportunity: Build an ML system to classify sentiment automatically

💰 Impact: Save 20+ hours/month + identify key satisfaction drivers

Why Does This Matter?

✅ Triage Negative Reviews: Auto-flag 1-2 star reviews for urgent response

📊 Track Trends: Monitor monthly sentiment score to measure improvement

🎯 Target Marketing: Extract positive keywords for campaigns

🔍 Root Cause Analysis: Identify common complaints across restaurants

⚡ Real-time Response: Respond to critical feedback within hours, not days

Our Dataset: Yelp Reviews

📊 Source: Hugging Face yelp_review_full (650,000+ reviews)

📝 Sample Used: 1,000 reviews for manageable training

🎯 Classes:

- Negative (1-2 stars): 404 reviews - 40.4%
- Positive (4-5 stars): 392 reviews - 39.2%
- Neutral (3 stars): 204 reviews - 20.4%

📈 Train/Test Split: 80% (800) / 20% (200)

Our Approach: TF-IDF + Machine Learning

1. Preprocessing: Lowercase, remove URLs, tokenize, lemmatize

2. Feature Extraction: Convert text to 5,000 TF-IDF features

3. Model Training: Train two models for comparison
   - Logistic Regression
   - Multinomial Naive Bayes

4. Evaluation: Test on unseen data with multiple metrics

5. Prediction: Classify new reviews as Positive/Negative/Neutral

Two Models Trained & Compared

Logistic Regression

✓ Interpretable

✓ Fast training

Accuracy: 67.5%

Precision: 54.2%

Recall: 67.5%

F1-Score: 60.1%

Naive Bayes (Best)

✓ Very fast (0.02 sec)

✓ Competitive accuracy

Accuracy: 68.0% 🏆

Precision: 54.1%

Recall: 68.0%

F1-Score: 60.2%

What Worked Well ✓

🟢 Positive Reviews: 86% correctly identified

→ Model easily recognizes 'great', 'excellent', 'amazing'

🔴 Negative Reviews: 86% correctly identified

→ Model easily recognizes 'terrible', 'bad', 'disappointing'

⚡ Fast Training: Naive Bayes trained in 0.02 seconds

📊 Clear Separation: Strong vs. weak sentiments naturally separable

✅ Overall Accuracy: 68% on completely unseen test data

The Challenge: Neutral Class ✗

🟡 Neutral Reviews (3 stars): 0% accuracy (all 41 samples misclassified)

❌ Why?

1. Class Imbalance: Only 204 neutral vs 400+ positive/negative
2. Ambiguous Language: '3-star' reviews mix good & bad
3. TF-IDF Limitation: Can't understand context ('good food, bad service')

📌 Root Cause: When training data has fewer examples, model defaults to common classes

💡 Silver Lining: This is a real, documented problem—not a bug, it's insight!

Real Examples: Successes vs. Failures

✅ SUCCESS

Text: "Love this place! Best restaurant in town."

Predicted: POSITIVE ✓

Confidence: 92%

Text: "Terrible experience. Cold food, rude staff."

Predicted: NEGATIVE ✓

Confidence: 87%

❌ FAILURE

Text: "Nice place, but food quality is mediocre."

Actual: NEUTRAL

Predicted: POSITIVE ✗

Confidence: 72%

[All 41 neutral reviews misclassified]

Key Learnings

🧠 Text Preprocessing Matters: Lemmatization + stopwords = big accuracy boost

🎯 Simple Models Work: Naive Bayes competitive with complex methods

⚖️ Class Imbalance is Real: Minority classes get ignored by default

📚 TF-IDF Limitations: Ignores word order, context, sarcasm

✍️ Honest Analysis > Perfect Metrics: Understanding failures is valuable

Next Steps: Future Improvements

🔧 Quick Wins:

→ Use SMOTE to balance classes (could reach 70-72% accuracy)

→ Implement class weights in model

🚀 Advanced:

→ BERT Transformers: Understand context & sarcasm (75%+ accuracy)

→ Aspect-Based Sentiment: Separate scores for food/service/ambiance

→ Collect more neutral examples: Better train the minority class

📈 Production:

→ Deploy with confidence thresholds

→ Build API for business integration

Questions?

Repository: https://github.com/KenDougg/ITAI-1378-Final-Project

Thanks to Professor Viswanatha Rao